Course Project Proposal — Automated Bootstrapping Placement for Encrypted Inference

Group. Xinghang ZENG (72656014), Chengyu BIE (72655619), Weijia ZHANG (72657103); contact email: 72656014@cityu-dg.edu.cn

Topic and why. We choose Topic 2, *Automated Bootstrapping Placement for Encrypted Inference*. In CKKS-style fully homomorphic encryption, every multiplicative operator consumes a ciphertext "level", and when levels run out an expensive Bootstrap must refresh the ciphertext. Deciding which edges of a network's computation graph carry Bootstraps (and Modswitches) determines whether the block is level-correct and how low its estimated latency is. We choose this topic because it is a pure search-and-optimisation problem — the core of this course: it can be formulated as constrained combinatorial optimisation over a DAG, and it needs no LLM API and no GPU. Everything runs on a laptop, and progress is measurable against objective baselines.

Tentative plan. Using the handout DAGs (the MLP block and the attention block, with level costs, fitted latency functions $c_v(l)$, and #cipher per edge), we decide for every edge (u,v) a chain $w_{u,v} \in \{\varnothing, B1, B2, M(\ell), B1\cdot M(\ell), B2\cdot M(\ell)\}$ with $0 \le \ell \le L_{max}$ (Bootstrap2 only on edges with #cipher 8 or 16; chains sharing a prefix insert and pay it once), minimising the estimated latency $f = \Sigma_v\, c_v(l_{v,in}) + \Sigma_u \Sigma_{p \in P_u}\, lat(p)$, subject to the level constraints (operator level costs, Bootstrap refresh levels, compatible input levels at multi-input nodes, operator validity ranges), for every input level $L_{init} \in \{0,\ldots,14\}$. Our method, in stages:

1. **Validation harness.** Reproduce the reference baselines with the handout scorer (evaluate_dag) before any optimisation, so that our reading of the rules is confirmed.
2. **Exact search.** A CSP / branch-and-bound formulation over level variables and chain choices, with forward checking; expected to solve the MLP block optimally for each L_init.
3. **Informed and local search.** A*-style state-space search over edge decisions with an admissible lower bound on the remaining latency, plus simulated annealing / local search with constraint-repair moves for the larger attention block.
4. **Optional extension.** Solve the joint attention+MLP DAG as a single placement problem.

Evaluation. Benchmark: the two provided blocks over the full range $L_{init} = 0$–14, scored by the reference scorer. Baselines to beat: the reference placements get_mlp_baseline_inserted() (≈28,130 ms, mildly dependent on L_{init}) and get_attn_baseline_inserted() (53,558.17 ms), and, for the extension, get_baseline() (81,688.82 ms). Success criterion: a feasible placement with strictly smaller f at every L_{init}; where the exact method applies we also report the optimality gap. We will report estimated latency, #B, depth, budget and utilisation for each L_{init}, and annotated DAG figures for representative values. No LLM is used in this project.